

Energy-Efficient Encoding for Multi-Level Cell PCM Using VQ-VAE-Based Masking

1st Sullivan Gleason

Computer Science Department
Southeast Missouri State University
Cape Girardeau, Missouri
sullivanpgleason@gmail.com

2nd Prince Kakadiya

Computer Science Department
California State University, Northridge
Northridge, California
prince-shaileshbhai.kakadiya.285@my.csun.edu

3rd Marjan Asadinia, IEEE Member

Computer Science Department
California State University, Northridge
Northridge, California
marjan.asadinia@csun.edu

Abstract—Phase-Change Memory (PCM) is emerging as a promising alternative to DRAM due to its high density, scalability, and non-volatility. However, write operations in PCM, particularly in multi-level cell (MLC) devices are energy-intensive and prone to reliability issues such as resistance drift, write-disturbance errors, and thermal crosstalk. In this work, we present a novel encoding framework based on Vector Quantized Variational Autoencoders (VQ-VAE) to generate optimized write masks that minimize high-energy state transitions. Our method incorporates PCM-aware cost functions, including write energy, error susceptibility, and write disturbance, into the VQ-VAE training process to intelligently balance energy efficiency and reliability. Experiments on large-scale memory-write data demonstrate substantial reductions in write energy and improvements in write-disturbance mitigation, while maintaining competitive reliability compared to unencoded payloads. These findings highlight the potential of learning-based encoding methods to extend PCM endurance while reducing energy consumption.

Index Terms—Phase-Change Memory (PCM), Multi-Level Cell (MLC), Write Energy Optimization, Vector Quantized Variational Autoencoder (VQ-VAE), Fuzzy Logic, Write Disturbance, Data Encoding.

I. INTRODUCTION

The increasing demand for high-density, energy-efficient memory technologies has driven significant interest in non-volatile memory (NVM) solutions. Among these, Phase-Change Memory (PCM) has gained attention due to its scalability, low standby power, and the ability to store multiple bits per cell. Unlike DRAM, which stores data as electrical charges, PCM exploits the resistance differences between amorphous and crystalline states of phase-change materials to represent data [1–3]. However, PCM write operations are inherently costly in terms of energy and endurance. These challenges become more severe in multi-level cell (MLC) PCM, where intermediate states (e.g., 01 and 10) consume up to an order of magnitude more energy compared to extreme states (00 and 11). Additionally, MLC PCM is more vulnerable to issues such as thermal crosstalk, write-disturbance errors, and cell degradation [4–15].

Conventional approaches, including wear-leveling and error-correction schemes, provide partial solutions but often fail to

simultaneously optimize energy efficiency and reliability. Recently, machine learning techniques have emerged as powerful tools for data encoding and compression, enabling adaptive optimization for specific hardware constraints. Autoencoders, in particular, have demonstrated strong capabilities in learning compact and meaningful data representations, making them well-suited for applications in memory systems [1]–[4].

In this paper, we propose a *VQ-VAE-based masking approach* to reduce write energy and improve endurance in MLC PCM. By quantizing latent embeddings into discrete codebook vectors, our model learns efficient write masks that minimize high-energy transitions while maintaining data integrity. We incorporate custom PCM-aware objective functions that account for write energy, error susceptibility, and write disturbance, enabling a holistic optimization framework. The main contributions of this work are:

- A novel VQ-VAE-based encoding strategy that reduces the frequency of high-energy intermediate states in MLC PCM.
- Custom loss functions designed to balance energy efficiency, error resistance, and disturbance mitigation.
- Extensive experimental evaluation, showing significant energy savings and improved performance over baseline unencoded payloads.

The remainder of this paper is organized as follows: Section II reviews background concepts related to MLC PCM and VQ-VAE models. Section III introduces our proposed methodology and PCM-aware cost functions. Section IV presents the experimental setup and evaluation results, while Section V concludes the paper and discusses future work.

II. MOTIVATION AND BACKGROUND

A. Multilevel-cell PCM

Multilevel-cell (MLC) PCM is a memory paradigm that allows for storing two or more bits of information on a single PCM cell by assigning each value to a specific resistance range in the phase-change material. This design increases memory density, but amplifies the likelihood of read/write based errors such as resistance drift [5], write-disturbance [6], [7], and thermal crosstalk [8].

This work focuses specifically on 2-bit MLC, and uses empirical findings from [9], as summarized in Table II. For the remainder of this paper, we refer to states 01 and 10 as *intermediate states*, and 00 and 11 as *extreme states*.

B. Vector Quantized Variational Autoencoders

Autoencoders are a widely used model in deep learning for efficiently learning compressed representations of data. Their structure consists of two main components; an *encoder* and a *decoder*. The encoder maps input data to a lower-dimensional latent space. The decoder then takes this latent vector and attempts to reconstruct the original input from this latent representation.

In 2017, Oord et al. [10] introduced the Vector Quantized Variational Autoencoder (VQ-VAE), as a discrete alternative to the standard autoencoder. VQ-VAEs include a *quantization* layer between the encoder and decoder, which maps continuous latent vectors to the closest entry in a learned codebook.

This discrete representation offers a key advantage for our application, as it dramatically reduces storage overhead by representing each latent as a small index rather than a full continuous vector. This property makes the VQ-VAE particularly well-suited for low-overhead data encoding with only a minor sacrifice in reconstruction quality.

C. Fuzzy Logic Relaxations

This work makes significant use of logical operators when defining and training our models. By default, these metrics are non-differentiable and block gradient flow during standard optimization via gradient descent. To overcome this limitation, we substitute discrete logic gates with continuous relaxations based on fuzzy logic.

Fuzzy logic extends binary truth values to the continuous interval $a \in [0, 1]$, enabling real-valued state representations. Table I details algebraic relaxations of commonly used logical operators following conventions introduced by Zadeh [11].

[!] **Note:** Throughout this work, we use the standard logical symbols ($\vee$, $\wedge$, $\oplus$, $\neg$) to denote their relaxed, differentiable counterparts unless otherwise specified.

D. Variable Notation

In this work, we will refer to the following variables frequently:

- D_{mem} $\leftarrow$ Data currently stored in memory
- D_{write} $\leftarrow$ Target payload to be written (unencoded)
- D_{mask} $\leftarrow$ Write mask
- D_{enc} $\leftarrow$ Encoded write payload ($D_{\text{write}} \oplus D_{\text{mask}}$)
- Δ $\leftarrow$ Bitwise soft difference ($D_{\text{mem}} \oplus D_{\text{enc}}$)

III. PROPOSED METHOD

This section presents the proposed methodology in detail. In our method, we train a Vector Quantized Variational Autoencoder (VQ-VAE) on three PCM-aware objective functions to infer bitmasks that minimize the overall write cost.

We begin by justifying our solution through an outline of common write issues in Phase-Change Memory that directly affect cell endurance and performance. We then introduce our model and training method at a high level, followed by detailed definitions of our custom PCM-aware loss algorithms.

A. Challenges in Efficient and Reliable PCM Writing

PCM write operations are inherently costly in terms of both energy efficiency and endurance. Excessive high-energy writes can accelerate cell wear and cause reliability issues such as write disturbance errors [6], [7], thermal crosstalk [8], and premature device degradation [5].

These issues are further compounded in Multilevel-cell (MLC) PCM. Table II [9] summarizes the average cost to program every state in 2-bit MLC. In particular, intermediate states 01 and 10 require $10 - 15\times$ more energy to write compared to 00 and 11. Reducing their occurrence is crucial for efficient energy writes.

In addition, such intermediate states require higher current and longer pulse durations, resulting in greater heat generation and a higher likelihood of write disturbance errors.

B. General Model Design

Our model is defined by three parameters:

- 1) $L \leftarrow$ Total Data block width
- 2) $N \leftarrow$ Codebook size (number of discrete embeddings)
- 3) $K \leftarrow$ Number of tokens predicted per input block

The high-level model architecture is described in Figure 1.

The encoder $\mathcal{E}$ takes inputs D_{mem} and D_{write} and produces K latent tokens. The quantizer $\mathcal{Q}$ then maps each token to the nearest vector in the codebook $\mathcal{G}$. The indices of the selected codebook vectors are stored as the encoding bits.

The decoder $\mathcal{D}$ then reconstructs the quantized vectors to produce a predicted XOR mask D_{mask} . During training, D_{mask} is softly XOR'ed with D_{write} to produce the final encoded payload D_{enc} , allowing gradients to flow through the XOR operation.

TABLE I
DIFFERENTIABLE APPROXIMATIONS OF STANDARD LOGIC GATES.

Logic Gate	Algebraic Approximation	Notation
XOR(A,B)	$A + B - 2(A \cdot B)$	$\oplus$
AND(A,B)	$A \cdot B$	$\wedge$
OR(A,B)	$A + B - (A \cdot B)$	$\vee$
NOT(A)	$1 - A$	$\neg$

TABLE II
WRITE ENERGY OF PROGRAMMING EVERY STATE [9].

State	Average Current	Pulse Duration	Average Energy Consumption
00	300 μA	40 ns	36 pJ
01	100 μA – 548 μA	220 ns – 520 ns	307 pJ
10	576 μA – 996 μA	220 ns – 520 ns	547 pJ
11	200 μA	150 ns	20 pJ

At evaluation time, the predicted mask D_{mask} is binarized, and the encoding is performed using the standard, discrete XOR operation.

The codebook indices are then prepended to D_{enc} and written to memory. During decoding, the mask can be regenerated using these indices, as each one maps directly to an entry in $\mathcal{G}$.

Although $\mathcal{D}$ is needed during training, it can optionally be omitted during inference by using a precomputed lookup table of all possible mask-index permutations.

The memory overhead introduced by this method is given by $\frac{\lceil \log_2 N \rceil \cdot K}{L}$. It follows that $U = L - \lceil \log_2 N \rceil \cdot K$ bits are available for actual storage. For optimal use of the encoding space, we recommend N be a power of 2.

C. MLC State Cost Tables

For each cell state in 2-bit PCM, we define three discrete cost tables corresponding to write energy (C_{ij}), error susceptibility (V_{ij}), and write aggression (T_{ij}), as summarized in Table III. Their derivations are described below.

- 1) **Write Energy Consumption:** Average Energy Consumption column from table II, normalized to C_{11} .
- 2) **Error Susceptibility:** A heuristic measure proportional to the relative amount of amorphous material in the cell. Amorphous states are known to have higher susceptibility to drift and thermal effects [12].
- 3) **Write Aggression (Write Disturbance Effect):** A heuristic estimate of the average thermal energy introduced when programming a given state. Derived from table II as a product of *Average Current* and *Pulse Duration*. For intermediate states (01 and 10), we use the mean of the two listed values. These values are then scaled by a factor of 10^{-5} for normalization.

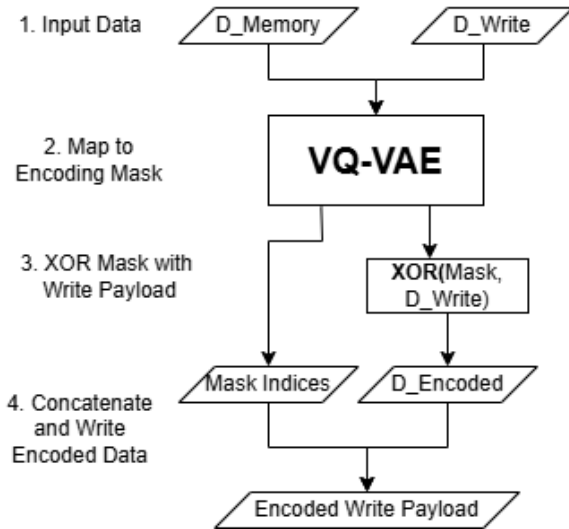


Fig. 1. Flowchart for encoding data in our method.

D. Objective Functions for PCM Write Metrics

We train our model on three distinct objective functions, each targeting specific issues in the PCM write process. By default, these metrics are non-differentiable and hinder standard gradient descent methods. To enable continuous optimization, we substitute discrete gates with fuzzy logic (II-C), mapping standard Boolean operations to smooth polynomial compositions as shown in table I.

First, we describe two helper functions:

- 1) **GETSTATEWEIGHTS** (Alg. 1) details the scheme for calculating per-state probabilities. In this algorithm, discrete states are given weightings based on their similarity to a pair of fuzzy input bits. Algorithm 1 forms the basis of the three objective functions, as it enables differentiable mapping of fuzzy-bit configurations to discrete cell states.
- 2) **GETCELLWISECOST** (Alg. 2) takes D_{enc} and a given cost table M as inputs. This algorithm then iterates over D_{enc} and applies M to each pair of fuzzy bits, using the probabilities calculated in GETSTATEWEIGHTS.

Next, we present Algorithms 3 to 5, which detail the three PCM-aware loss metrics that we used to train our models.

1) **Write Energy Cost:** Algorithm 3 details our write energy objective function. This algorithm takes Δ , D_{enc} , and the write cost table C as inputs, and outputs the total energy write cost $\text{Cost}_{\text{write}}$. The algorithm first applies C to the payload D_{enc} , to calculate the cell-wise write cost. These values are then scaled by soft bitwise differences between the write payload and the cell state. Finally, we apply a sum operation to calculate $\text{Cost}_{\text{write}}$.

This algorithm implicitly penalizes high proportions of intermediate states, as their write-energy costs are significantly higher than those of extreme states, while ignoring the cost of cells that do not need to be overwritten.

2) **Error Susceptibility:** Algorithm 4 penalizes high error-prone write patterns. Specifically, this algorithm penalizes amorphous cell states, which tend to exhibit greater error susceptibility due to their disordered structure [12]. Algorithm 4 simply applies the error cost table V to D_{enc} , and returns the sum Cost_{err} . This function only evaluates the final write state, so it does not need information on Δ .

3) **Write Disturbance:** Algorithm 5 discourages aggressive writes near high-error-prone cells. This algorithm takes Δ , D_{enc} , the error cost table V , and the WDE cost table T as inputs.

First, the cell-wise error cost Cost_{err} and the cell-wise write aggression cost Cost_{agg} are computed. The algorithm then

TABLE III
DERIVED COST TABLES FOR EVERY STATE.

State	Energy Consumption	Error Susceptibility	Write Aggression
00	1.8	1.0	0.12
01	15.35	0.7	1.20
10	27.35	0.4	2.91
11	1.0	0.0	0.30

iterates over each pair of bits in Δ and computes the write weighting w_{diff} .

Since write disturbance is triggered by active programming, Cost_{agg} is scaled by w_{diff} .

Cost_{err} is only applicable to dormant (unwritten) cells (as they may be disturbed by neighboring writes) so it is scaled by the inverse weighting $\neg w_{\text{diff}}$.

Next, to quantify the write-disturbance effect on cells from their neighbors, we apply a 1D convolution to Cost_{agg} using the filter $[1, 0, 1]$. This accumulates the aggression of a given cell's immediate neighbors, excluding itself. The result is then scaled element-wise by Cost_{err} cost to produce $\text{Cost}_{\text{cellwise}}$.

Finally, Cost_{wde} is returned as the sum of $\text{Cost}_{\text{cellwise}}$.

Algorithm 1 GETSTATEWEIGHTS

Input: b_0, b_1 $\triangleright$ Bits from encoded data
Output: $(w_{00}, w_{01}, w_{10}, w_{11})$
 $w_{00} \leftarrow \neg(b_0 \vee b_1)$
 $w_{01} \leftarrow (\neg b_0) \wedge b_1$
 $w_{10} \leftarrow b_0 \wedge (\neg b_1)$
 $w_{11} \leftarrow b_0 \wedge b_1$
return $(w_{00}, w_{01}, w_{10}, w_{11})$

Algorithm 2 GETCELLWISECOST

Input: D_{enc}, M $\triangleright$ Encoded Payload, cost table
Output: CellCosts
 $N_{\text{cells}} \leftarrow \text{len}(D_{\text{enc}})/2$ $\triangleright$ Number of 2-bit cells
CellCosts $\leftarrow [0, 0, \dots, 0]$ of length N_{cells}
 $k \leftarrow 0$
while $k \leq N_{\text{cells}}$ **do**
 $(b_0, b_1) \leftarrow D_{\text{enc}}[k], D_{\text{enc}}[k+1]$ $\triangleright$ Extract 2-bit slice
 $(w_{00}, w_{01}, w_{10}, w_{11}) \leftarrow \text{GETSTATEWEIGHTS}(b_0, b_1)$
CellCosts $[k/2] \leftarrow \sum_{s \in \{00, 01, 10, 11\}} w_s \cdot M_s$
 $k \leftarrow k + 2$
end while
return CellCosts

IV. EXPERIMENTAL SETUP AND RESULTS

This section outlines our experimental methodology. We first detail the process of collecting and organizing the data for model training and evaluation. We then describe the configurations used in our experiment, including model design, encoding sizes, and block widths.

We then evaluate our proposed method using unencoded payloads as a baseline. Finally, we discuss the trade-offs between specific model choices and how they affect overall performance.

A. Data Collection and Organization

We sourced data for this study from the NVMain simulator [13], [14], [15], [16], [17], [18], [19] a respected and reliable simulator for non-volatile memory technologies. The data files were collected using real application workloads, such

Algorithm 3 Total Write Energy Cost

Input: $\Delta, D_{\text{enc}}, C$ $\triangleright$ Bitwise difference and encoded write payload, Write cost table
Output: $\text{Cost}_{\text{write}}$ $\triangleright$ Total Write Energy Cost
 $\text{Cost}_{\text{cellwise}} \leftarrow \text{GETCELLWISECOST}(D_{\text{enc}}, C)$
 $N_{\text{cells}} \leftarrow \text{len}(D_{\text{enc}})/2$ $\triangleright$ Number of 2-bit cells
 $k \leftarrow 0$
while $k \leq N_{\text{cells}}$ **do**
 $w_{\text{diff}} \leftarrow \Delta_k \vee \Delta_{k+1}$ $\triangleright$ Write weight
 $\text{Cost}_{\text{cellwise}}[k/2] \leftarrow \text{Cost}_{\text{cellwise}}[k/2] \cdot w_{\text{diff}}$
 $k \leftarrow k + 2$
end while
 $\text{Cost}_{\text{write}} \leftarrow \sum_{i=0}^{N_{\text{cells}}-1} \text{Cost}_{\text{cellwise}}[i]$
return $\text{Cost}_{\text{write}}$

Algorithm 4 Error Susceptibility Cost

Input: D_{enc}, V $\triangleright$ Encoded write payload, Error cost table
Output: Cost_{err} $\triangleright$ Total Error-susceptibility Cost
 $\text{Cost}_{\text{cellwise}} \leftarrow \text{GETCELLWISECOST}(D_{\text{enc}}, V)$
 $N_{\text{cells}} \leftarrow \text{len}(D_{\text{enc}})/2$ $\triangleright$ Number of 2-bit cells
 $\text{Cost}_{\text{err}} \leftarrow \sum_{i=0}^{N_{\text{cells}}-1} \text{Cost}_{\text{cellwise}}[i]$
return Cost_{err}

Algorithm 5 Write Disturbance Cost

Input: $\Delta, D_{\text{enc}}, V, T$
Output: Cost_{wde} $\triangleright$ Total Write Disturbance Cost
 $\text{Cost}_{\text{err}} \leftarrow \text{GETCELLWISECOST}(D_{\text{enc}}, V)$
 $\text{Cost}_{\text{agg}} \leftarrow \text{GETCELLWISECOST}(D_{\text{enc}}, T)$ $\triangleright$ Aggression
 $N_{\text{cells}} \leftarrow \text{len}(D_{\text{enc}})/2$ $\triangleright$ Number of 2-bit cells
 $k \leftarrow 0$
while $k \leq N_{\text{cells}}$ **do**
 $w_{\text{diff}} \leftarrow \Delta_k \vee \Delta_{k+1}$ $\triangleright$ Write weight
 $\text{Cost}_{\text{err}}[k/2] \leftarrow \text{Cost}_{\text{err}}[k/2] \cdot \neg w_{\text{diff}}$
 $\text{Cost}_{\text{agg}}[k/2] \leftarrow \text{Cost}_{\text{agg}}[k/2] \cdot w_{\text{diff}}$
 $k \leftarrow k + 2$
end while
NeighborFilter $\leftarrow [1, 0, 1]$
VictimWeights $\leftarrow \text{Cost}_{\text{agg}} * \text{NeighborFilter}$ $\triangleright$ Convolution
 $\text{Cost}_{\text{cellwise}} \leftarrow \text{Cost}_{\text{err}} \cdot \text{VictimWeights}$ $\triangleright$ Element-wise multiplication
 $\text{Cost}_{\text{wde}} \leftarrow \sum_{i=0}^{N_{\text{cells}}-1} \text{Cost}_{\text{cellwise}}[i]$
return Cost_{wde}

as those from SPEC CPU [20], and PARSEC benchmarks [21], to closely reflect realistic memory access patterns. By incorporating representative data access behaviors and diverse execution characteristics, these traces provide a faithful simulation of real-world scenarios, thereby improving the accuracy and reliability of our evaluations conducted with NVMain. It is also notable that NVMain is a versatile, cycle-accurate simulator that models both traditional DRAM and emerging NVM technologies such as PCM. It provides detailed insights into memory timing behavior, energy consumption, and NVM-specific characteristics like limited write endurance and multi-level cell structures. Additionally, NVMain supports hybrid memory architectures, fine-grained parallelism at the bank and subarray levels, and customizable memory controllers and address mapping schemes. Its ability to model key PCM features including asymmetric read/write latencies, high write energy, and endurance constraint makes it particularly suitable for evaluating PCM optimization strategies.

To simulate memory write behavior, we partitioned the dataset two equal halves: one representing the current memory contents and the other representing the new write payload. For our experiments, we used 800,000 sample data blocks, which produces 400,000 unique pairs of memory and write payloads. We then partitioned the data with an 80/20 split between training and testing sets.

For each model configuration, the dataset was cropped to account for the number of encoding bits used. Specifically, for a block width L , codebook size N and a number of predicted tokens K , we cropped the memory and write portions of the dataset to both have a width of $U = L - \lceil \log_2 N \rceil \cdot K$, as described in section III-B. This ensures that, after encoding, the combined input and encoding bits always occupy a fixed width of L .

B. VQ-VAE architecture

Our VQ-VAE model consists of three main components. Their descriptions are detailed below, with specific model layers and parameters summarized in table IV:

1) *Encoder ($\mathcal{E}$):* The encoder begins with two identical encoding blocks $\mathcal{B}_1$ and $\mathcal{B}_2$. $\mathcal{B}_1$ encodes the memory D_{mem} , while $\mathcal{B}_2$ encodes the write payload D_{write} . This parallel structure allows for isolated processing of D_{mem} and D_{write} , helping $\mathcal{E}$ learn a deeper representation of the input data. After this separate encoding step, the results are concatenated as input to a final joint encoding block $\mathcal{J}$, which produces K input tokens of length LatentDim. for all experiments, LatentDim = 128.

2) *Quantizer ($\mathcal{Q}$):* The quantizer takes the K input tokens and maps each to the nearest entry in its codebook.

To aid in training, we adopted the exponential moving average (EMA) update rule, as outlined in [10]. We also included a diversity function, which penalized skewed code distributions. Formally:

$$\text{Cost}_{\text{diverse}} = - \sum_{i=1}^N p_i \log(p_i + \epsilon) \quad (1)$$

Where p_i is the normalized usage of codebook entry i , N is the codebook size, and ϵ is a small constant.

Finally, we routinely removed unused codes, replacing them with codes similar to the most frequently used codes in the codebook.

3) *Decoder ($\mathcal{D}$):* The decoder then flattens the codebook vectors chosen by $\mathcal{Q}$, and applies transformations similar to $\mathcal{E}$, but in reverse order, to produce D_{mask} .

4) *Data Encoding:* D_{mask} is then XOR'ed with the input string D_{write} to produce D_{enc} . In evaluation, D_{mask} rounded to binary values before encoding to ensure binary mask outputs, rather than standard floating-point output of the soft XOR operation. This rounding also improves the results for each cost metric by 2-5%.

C. Model Training

All VQ-VAE configurations in this study were trained for 20 epochs with a batch size of 1024. All configurations also used the Adam optimizer, with a learning rate of 0.0005.

For the first two epochs, the model was trained to minimize the mean square error between the reconstructed data D_{recon} and D_{write} , as well as $\text{Cost}_{\text{diverse}}$. This allowed the model to learn the data distribution thoroughly before applying the more complex PCM-aware cost functions.

For the next five epochs, the PCM-aware cost functions were annealed into the total loss, while the mean square error was slowly removed.

By the eighth epoch, the PCM objective functions are in full effect, and MSE is completely removed. The final composite loss function is as follows:

$$\begin{aligned} \text{Cost}_{\text{total}} = & (w_{\text{diverse}} \cdot \text{Cost}_{\text{diverse}}) + (w_{\text{write}} \cdot \text{Cost}_{\text{write}}) \\ & + (w_{\text{err}} \cdot \text{Cost}_{\text{err}}) + (w_{\text{wde}} \cdot \text{Cost}_{\text{wde}}) \end{aligned} \quad (2)$$

Where w_{diverse} , w_{write} , w_{err} , and w_{wde} are the weightings of their respective cost functions (III-D).

D. General Results

To find optimal encoding configurations for a general model, we conducted a large grid search of block width $L \in \{128, 256, 512, 1024\}$, codebook size $N \in \{4, 16, 64, 128, 256\}$ and number of predicted tokens $K \in \{1, 2, 4, 6, 8\}$.

For preliminary testing, loss weights were kept static at the following values:

- $w_{\text{diverse}} \leftarrow 0.01$
- $w_{\text{write}} \leftarrow 0.01$
- $w_{\text{err}} \leftarrow 0.01$
- $w_{\text{wde}} \leftarrow 0.001$

These weights were not tuned for any specific use case, but rather to reflect the general objective of prioritizing low write energy while maintaining awareness of error susceptibility and write disturbance.

We track three result metrics, *write energy improvement*, *error susceptibility improvement*, and *WDE improvement*. These

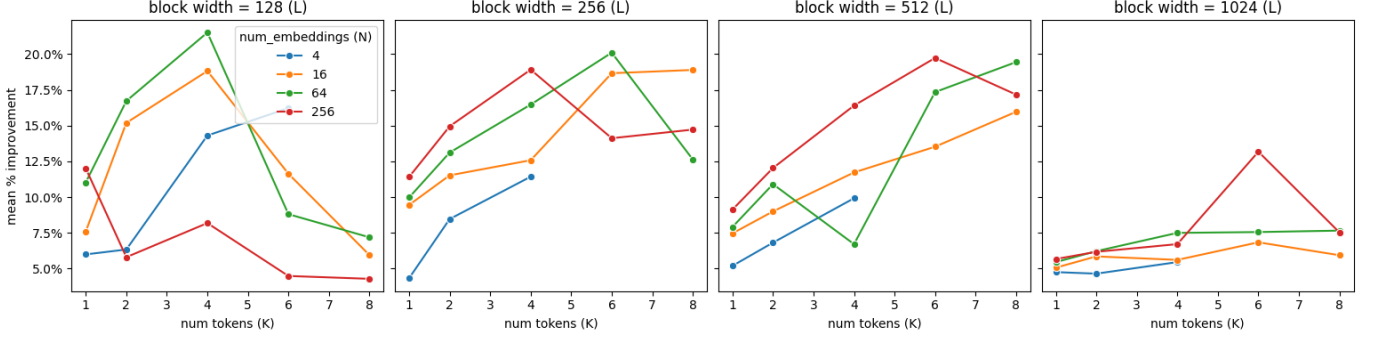


Fig. 2. Average mean improvement percentage across varying block widths, codebook sizes, and amounts of tokens.

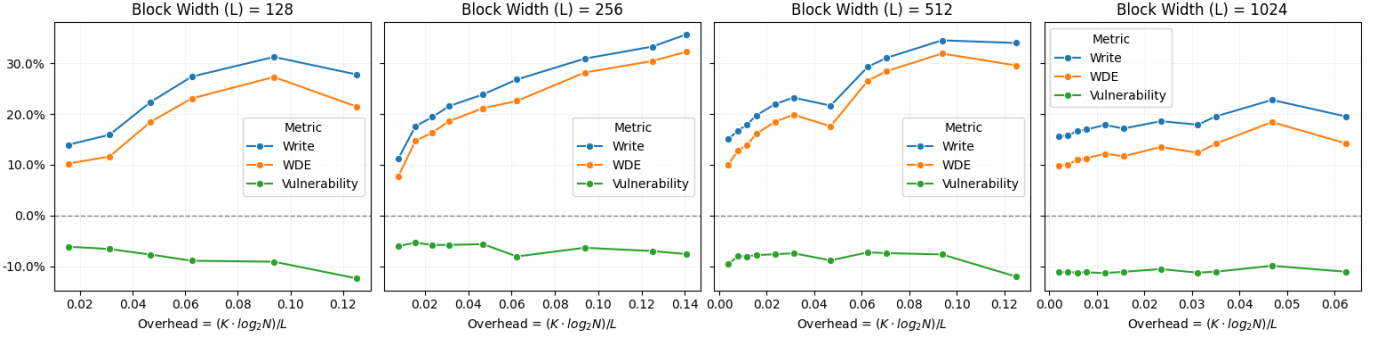


Fig. 3. Average % improvement of performance metrics vs. memory overhead for the encoding scheme (overhead $\leq 15\%$).

are calculated as the percent improvement of their respective cost functions over the baseline (unencoded) payload.

Our results across all configurations show an average write energy improvement of 24.1%, WDE improvement of 18.9%, and an error improvement of -11.1%.

While write energy and WDE metrics saw significant improvement, error susceptibility worsened. We consider this to be due to the compounding effects of write energy and WDE mitigation outweighing the weaker error susceptibility loss. In fact, across *all* encoding configurations, error susceptibility degraded. In section IV-E, we explore parameter tunings that reach meaningful improvements in all three metrics.

Figure 2 summarizes the results in further detail. For each configuration, we display the *mean discrete improvement*, calculated the mean of the three result metrics listed above. For $L = 128$, it is clear that four tokens produced the most optimal results, while for $L = 256$ and 512, eight tokens produces the best results. For $L = 1024$, moderate results are seen across all configurations, excluding a noticeable spike at $K = 6$, $N = 256$.

Figure 3 displays the results organized by the improvement of each metric as a function of encoding overhead. The graph is truncated at Overhead ≤ 0.15 . After this mark, performance plateaus and begins to decline, likely indicating where the model reached capacity. We also include a dashed line at $y=0$ to indicate the unencoded baseline.

Figure 3 shows that for all block widths, 20% write energy

improvement is attainable with less than 5% encoding overhead. For $L = 512$, this mark is surpassed with only 1.5% overhead.

A similar trend is observed for write disturbance, which follows the same general curve as write energy, albeit slightly lower. Both write energy and write disturbance also reach 30% improvement with an overhead $\leq 10\%$.

In contrast, error susceptibility consistently worsens with increasing overhead, dropping from approximately -5% to -10% across overhead configurations under these specific parameter settings.

E. Effects of Weight Tuning

Having established a general-purpose configuration in earlier experiments, we now investigate the effect of tuning the individual loss weights defined in eq. (2) for specific use cases.

This section focuses on how varying the relative importance of write energy, error susceptibility, and WDE cost components influences performance of each metric specifically. Our goal is to better understand the tradeoffs between these competing objectives and identify more specialized configurations.

Our results are from grid search of write energy weights $w_{\text{write}} \in \{0.0, 0.002, 0.004, 0.006, 0.008, 0.01\}$, WDE weights $w_{\text{wde}} \in \{0.0, 0.02, 0.04, 0.06, 0.08, 0.1\}$, and error susceptibility weights $w_{\text{err}} \in \{0.0, 0.2, 0.4, 0.6, 0.8, 1.0\}$. The model configuration was held static at $L = 128$, $N = 64$, $K = 2$, which has an encoding overhead of 9.4%.

TABLE IV
MODEL ARCHITECTURE COMPONENTS AND LAYER SPECIFICATIONS.

Module	Layer	Details
Memory Encoder, Write Encoder	Reshape	$(U, 1)$
	Conv1D	64 filters, kernel 4, stride 4, ReLU
	Conv1D	32 filters, kernel 4, stride 4, ReLU
	Dense	$U/2$, ReLU
Joint Encoder	Dense	U , ReLU
	Dense	U , ReLU
	Dense + Reshape	$(K, 128)$
Quantizer	EMAQuantizer	K embeddings
		$D = \text{latent_dim}$
		Commitment Cost = 0.1
Decoder	Flatten + Dense	$U \times 16$, ReLU
	Reshape	$(U, 16)$
	Conv1D	64 filters, kernel 4, ReLU
	Conv1D	1 filter, kernel 4, sigmoid
	Reshape	(U)
Optimizer / Training		Adam optimizer (lr = 0.0005) Batch size = 1024 Epochs = 20 Loss = (2)

We evaluated the best-performing configurations for each objective individually. Figure 4 displays the best weight configuration per metric in our testing. Table V provides a summary of these configurations and their respective tradeoffs.

The model that maximized write energy improvement used weights $(w_{\text{write}}, w_{\text{wde}}, w_{\text{err}}) = (0.01, 0.02, 0.00)$ and achieved a 31.12% gain. For write disturbance, the optimal configuration $(0.01, 0.10, 0.00)$ resulted in a 30.38% improvement. The best configuration for reducing error susceptibility was $(0.00, 0.06, 0.20)$, improving error resistance by 19.77%. Finally, the configuration that achieved the highest mean performance across all metrics, $(0.008, 0.02, 0.40)$, produced a balanced overall improvement of 20.04%.

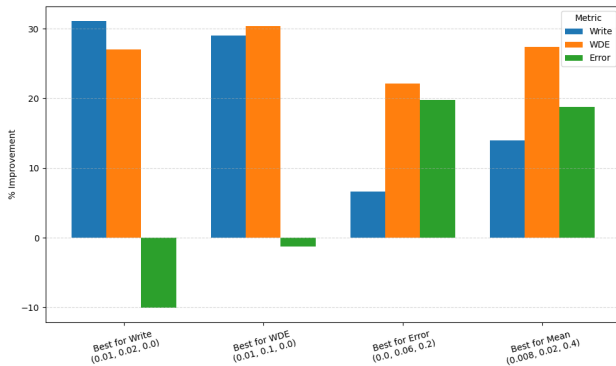


Fig. 4. Improvements from weight configurations that maximize each objective. Configurations are formatted as $(w_{\text{write}}, w_{\text{wde}}, w_{\text{err}})$

These results highlight the significant overlap that each of the three PCM cost functions exhibit, while still being distinct enough to warrant specific configurations for focused improvements.

The results also show the conflicting nature between error susceptibility and the other two cost functions, which is due in part to the specific properties of each PCM state, as shown in Table III. Both the write energy and WDE penalties favor extreme states $(00, 11)$, while general error susceptibility favors more crystalline states $(10, 11)$. This slight discrepancy may be enough to make these functions generally incompatible without specific weight tuning. This conclusion reflects the results from our primary testing (Fig. 3), and also explains why write Energy and WDE improvement both see their best results with $w_{\text{err}} = 0.0$.

These results highlight the significant overlap that each of the three PCM cost functions exhibit, while still being distinct enough to warrant specific configurations for focused improvements.

The results also show the conflicting nature between error susceptibility and the other two cost functions, which is due in part to the specific penalty weightings for each PCM state, as shown in Table III. In these penalty weightings, both the write energy and WDE penalties favor extreme states $(00, 11)$, while general error susceptibility favors more crystalline states $(10, 11)$. This slight discrepancy may be enough to make these functions generally incompatible without specific weight tuning. This conclusion reflects the results from our primary testing (Fig. 3), and also explains why write Energy and WDE improvement both see their best results with $w_{\text{err}} = 0.0$.

TABLE V
IMPROVEMENT METRICS AND WEIGHT CONFIGURATIONS FOR THE BEST-PERFORMING SETUP UNDER EACH OBJECTIVE.

Objective	Weight Config	Write Imp.	WDE Imp.	Error Imp.	Mean Imp.
Best for Write	(0.01, 0.02, 0.00)	31.12	27.05	-10.07	16.03
Best for WDE	(0.01, 0.10, 0.00)	29.03	30.38	-1.29	19.37
Best for Error	(0.00, 0.06, 0.20)	6.65	22.09	19.77	16.17
Best for Mean	(0.008, 0.02, 0.40)	14.00	27.38	18.73	20.04

F. Analysis of codes

To ensure a robust coverage of the latent space, we examine the embeddings stored in the quantizer’s codebook.

We begin by analyzing codebook usage, a valuable metric that reflects how effectively the quantizer maps encoded vectors to entries in the codebook.

In an ideally uniform quantizer, each code has a $\frac{K}{N}$ probability of being selected at any given time. For this testing, we used model configurations $L = 256, N = 256, K = 4$ with 40,000 test cases. Other model configurations exhibited similar distributions and are omitted for brevity.

Assuming uniform probabilities, each code should appear in $\frac{4}{256} \approx 1.5\%$ encoding sequences.

Figure 5 details the distribution per code for the above model, with a horizontal line at the expected average. The spread is relatively tight, with a mean of 1.56% and a standard

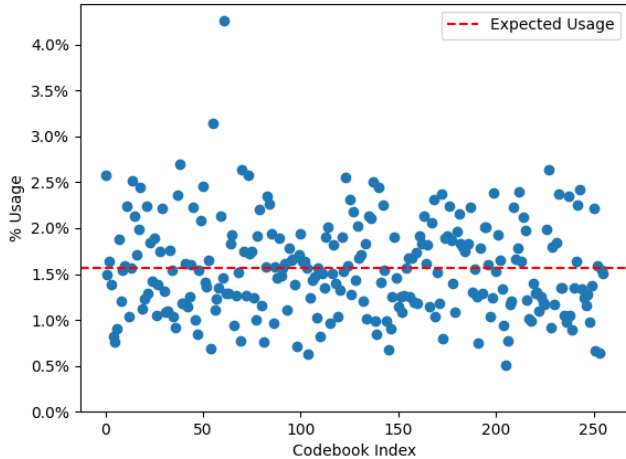


Fig. 5. Percent usage for each codebook vector for model $L = 256, N = 256, K = 4$ plotted against expected usage.

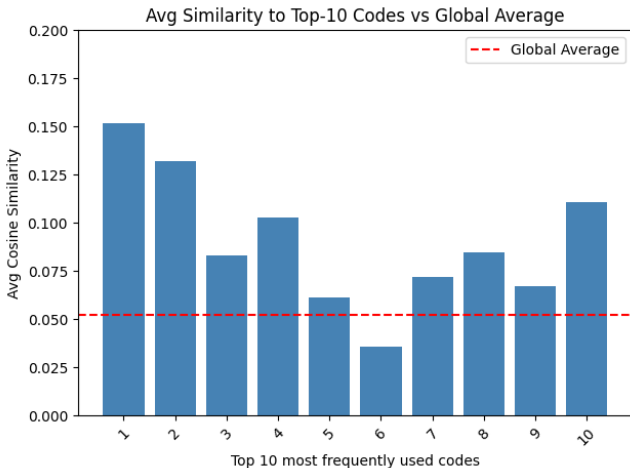


Fig. 6. Cosine similarity to codebook vectors to top 10 codes in codebook for $L = 256, N = 256, K = 4$ plotted against average similarity.

deviation of 0.52%. suggesting a healthy coverage of the latent space.

Next, we examine the similarities between codes in the codebook to ensure there is no significant redundancy among codes.

Figure 6 displays the average cosine similarity of the 10 most used codes with respect to all other codes. The average pairwise similarity across the entire codebook is also plotted as a horizontal dashed line for reference.

The average cosine similarity between any two codes in this embedding space was 0.052, meaning that codes are generally orthogonal with little redundancy. 9 of the top 10 codes are above this average, but none have an average similarity high enough to cause concern.

V. CONCLUSION AND FUTURE DIRECTION

In this paper, we demonstrated that VQ-VAE-based encoding is a powerful tool for mitigating the inherent challenges of MLC PCM. Our method achieves notable reductions in write energy and write-disturbance errors while maintaining competitive reliability, making it a promising alternative to conventional static encoding or wear-leveling approaches. By embedding PCM-specific characteristics such as energy asymmetry between intermediate and extreme states into the training process, our model adaptively learns encoding patterns that optimize both endurance and energy efficiency.

A key insight from our experiments is the trade-off between error susceptibility and energy optimization, which highlights the importance of fine-tuning loss function weights for specific deployment scenarios. Our comprehensive analysis of codebook usage and latent space confirms the robustness of the learned encoding space, demonstrating balanced code utilization and minimal redundancy.

Overall, this work not only showcases the feasibility of leveraging deep learning and VQ-VAE models for hardware-aware encoding but also provides a blueprint for designing future adaptive memory encoding schemes that dynamically optimize multiple reliability and performance objectives.

Future work can focus on developing adaptive, real-time encoding strategies that dynamically adjust to workload behavior, as well as integrating hybrid schemes that combine VQ-VAE encoding with error correction mechanisms for improved reliability. Further exploration of hardware-aware implementations, such as FPGA or ASIC prototypes, will provide insights into the practical trade-offs between latency, energy consumption, and area overhead.

VI. ACKNOWLEDGMENT

This work was supported by the National Science Foundation (NSF) under Grant Nos. [CNS-2244391] and [2318553]. We gratefully acknowledge the NSF for their financial support.

REFERENCES

- [1] Y. Lim, D. Kim, and J. Kim, "Sc: Efficient error correction codes for mlc pcm," in *2023 20th International SoC Design Conference (ISOCC)*, pp. 303–304, 2023.
- [2] M. Shi, P. Mo, and J. Liu, "Deep neural network for accurate and efficient atomistic modeling of phase change memory," *IEEE Electron Device Letters*, vol. 41, pp. 365–368, Mar. 2020.
- [3] K. Fatemi and Masoud, "A power-aware reinforcement learning technique for memory allocation in real-time embedded systems," <https://prism.ucalgary.ca>, 2022. Available at: <https://prism.ucalgary.ca>.
- [4] S. Song, A. Das, O. Mutlu, and N. Kandasamy, "Improving phase change memory performance with data content aware access," *arXiv preprint arXiv:2006.10991*, June 2020.
- [5] J. Li, B. Luan, and C. Lam, "Resistance drift in phase change memory," in *2012 IEEE International Reliability Physics Symposium (IRPS)*, pp. 6C.1.1–6C.1.6, 2012.
- [6] M. Kim, H. Lee, H. Kim, and H.-J. Lee, "Wl-wd: Wear-leveling solution to mitigate write disturbance errors for phase-change memory," *IEEE Access*, vol. 10, pp. 11420–11431, 2022.
- [7] J. Jang, W. Shin, J. Choi, Y. Kim, and L.-S. Kim, "Sparse-insertion write cache to mitigate write disturbance errors in phase change memory," *IEEE Transactions on Computers*, vol. 68, no. 5, pp. 752–764, 2019.

- [8] S. M. Seyedzadeh, A. K. Jones, and R. Melhem, "Improving sustainability through disturbance crosstalk mitigation in deeply scaled phase-change memory," in *2018 Ninth International Green and Sustainable Computing Conference (IGSC)*, pp. 1–8, 2018.
- [9] J. Wang, X. Dong, G. Sun, D. Niu, and Y. Xie, "Energy-efficient multi-level cell phase-change memory system with data encoding," in *2011 IEEE 29th International Conference on Computer Design (ICCD)*, pp. 175–182, 2011.
- [10] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, "Neural discrete representation learning," in *Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS'17*, (Red Hook, NY, USA), p. 6309–6318, Curran Associates Inc., 2017.
- [11] L. Zadeh, "Fuzzy sets," *Information and Control*, vol. 8, no. 3, pp. 338–353, 1965.
- [12] S. J. Ahn, Y. Song, H. Jeong, B. Kim, Y.-S. Kang, D.-H. Ahn, Y. Kwon, S. W. Nam, G. Jeong, H. Kang, and C. Chung, "Reliability perspectives for high density pram manufacturing," in *2011 International Electron Devices Meeting*, pp. 12.6.1–12.6.4, 2011.
- [13] M. Poremba, T. Zhang, and Y. Xie, "Nvmain 2.0: A user-friendly memory simulator to model (non-)volatile memory systems," *IEEE Computer Architecture Letters*, vol. 14, pp. 140–143, July 2015.
- [14] M. Desai, R. Quinn, and M. Asadinia, "SMART-WRITE: Adaptive Learning-Based Write Energy Optimization for Phase Change Memory," in *Proceedings of the 2025 IEEE 15th Annual Computing and Communication Workshop and Conference (CCWC)*, pp. 640–648, Jan. 2025.
- [15] J. Ekoniak, A. Rumale, and M. Asadinia, "ML-PreP: Machine Learning Based Error Prediction for Phase Change Memory," in *Proceedings of the 2025 IEEE 15th Annual Computing and Communication Workshop and Conference (CCWC)*, pp. 909–917, Jan. 2025.
- [16] M. Desai, R. Quinn, and M. Asadinia, "ML-PCM: Machine Learning Technique for Write Optimization in Phase Change Memory (PCM)," in *Communications in Computer and Information Science*, pp. 207–219, Jan. 2025.
- [17] A. Rumale, J. Ekoniak, and M. Asadinia, "PENN: Predicting Errors Using Neural Network in Phase Change Memory," in *Communications in Computer and Information Science*, pp. 278–293, Jan. 2025.
- [18] M. Desai, A. Rumale, M. Asadinia, and S. Bogle, "WIRE: Write Energy Reduction via Encoding in Phase Change Main Memories (PCM)," in *Lecture Notes in Networks and Systems*, pp. 599–615, 2024.
- [19] M. Desai, A. Rumale, and M. Asadinia, "HART: A Hybrid Addressing Scheme for Self-Balancing Binary Search Trees in Phase Change Memory (PCM)," in *Proceedings of the 2025 IEEE 4th International Conference on Computing and Machine Intelligence (ICMI)*, pp. 1–6, Apr. 2025.
- [20] C. D. Spradling, "Spec cpu2006 benchmark tools," *SIGARCH Comput. Archit. News*, vol. 35, p. 130–134, Mar. 2007.
- [21] C. Bienia, S. Kumar, J. P. Singh, and K. Li, "The parsec benchmark suite: characterization and architectural implications," in *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques, PACT '08*, (New York, NY, USA), p. 72–81, Association for Computing Machinery, 2008.